Funkcje biblioteczne

C++ | Programowanie aplikacji | technik programista | INF.04

Lekcja 6



10

Funkcje biblioteczne

Język C++, podobnie jak inne języki programowania, jest wyposażony w szereg przydatnych w praktyce **funkcji predefiniowanych** (ang. *predefined functions*). Funkcje te są zgrupowane w **standardowej bibliotece C++** (ang. *C++ Standard Library*). Dlatego też nazywa się je **funkcjami standardowymi** (ang. *standard functions*).

Biblioteka standardowa C++ składa się z dwóch części:

- **biblioteki funkcji standardowych** (ang. *Standard Function Library*),
- **zorientowanej obiektowo biblioteki klas** (ang. *Object Oriented Class Library*).

Tym samym funkcje standardowe można podzielić na dwie grupy:

- funkcje odziedziczone po języku C, zgrupowane w bibliotece funkcji standardowych,
- funkcje członkowskie klas wchodzących w skład biblioteki klas.

UWAGA

Klasy jako elementy języka C++ zostały omówione szczegółowo w rozdziale 11. podręcznika.

Funkcje odziedziczone po języku C są funkcjami autonomicznymi — nie są zależne ani jedna od drugiej, ani też od żadnej klasy. Funkcje te są zgrupowane w różnych „bibliotekach", które z punktu widzenia programisty są **zbiorami nagłówkowymi** (ang. *header files*). Tym samym, aby można było skorzystać z określonej funkcji, należy dołączyć do programu — za pomocą dyrektywy `#include` — bibliotekę, w której ta funkcja została zadeklarowana.

Natomiast funkcje standardowe należące do biblioteki klas to funkcje członkowskie określonej klasy, która została tam zdefiniowana. Dostęp do nich uzyskuje się w analogiczny sposób jak do funkcji odziedziczonych po języku C — przez dołączenie za pomocą dyrektywy `#include` zbioru nagłówkowego zawierającego deklaracje niezbędnych zasobów.

Tematyka wykorzystania funkcji standardowych była już poruszana w podręczniku. W szczególności dotyczy to funkcji napisowych i łańcuchowych, pozwalających wykonywać różne operacje na, odpowiednio, C-napisach oraz łańcuchach znaków typu `string`. Funkcje na-

255

Cele lekcji i kontekst

pojęcie biblioteki | nagłówki | funkcje gotowe | dobór funkcji

- wyjaśnisz, czym są funkcje predefiniowane i biblioteka standardowa C++
- dobierzesz właściwy plik nagłówkowy: <cmath>, <cctype>, <cstdlib>, <cstdio>, <ctime>
- zastosujesz funkcje matematyczne, znakowe, konwersji, wejścia/wyjścia i losowania
- porównasz zapis w stylu C i C++ oraz przeanalizujesz gotowy kod
- utrwalisz poprawne użycie dyrektywy #include i dokumentacji funkcji

Efekty kształcenia

Uczeń tworzy i analizuje programy w C++, korzysta z dokumentacji technicznej, dobiera elementy biblioteki standardowej oraz testuje działanie gotowych funkcji w praktycznych zadaniach.

Biblioteka standardowa C++

Biblioteka standardowa C++ to zestaw gotowych elementów języka. Funkcje z biblioteki są już napisane, przetestowane i dostępne po dołączeniu właściwego nagłówka.




```
#include <iostream>
#include <cmath>
#include <cctype>
#include <cstdlib>
#include <cstdio>
#include <ctime>
```

- funkcje odziedziczone z C są zwykle autonomiczne
- funkcje klasowe wywołuje się jako metody obiektu
- nowy skrót: STL = Standard Template Library, część biblioteki standardowej C++

Funkcje matematyczne - <cmath>

Nagłówek <cmath> daje dostęp m.in. do działań na liczbach, pierwiastków, potęg, logarytmów, trygonometrii i zaokrągleń.

```
sqrt(4)      // 2
pow(3, 2)    // 9
round(1.55)  // 2
ceil(1.55)   // 2
floor(1.55)  // 1
sin(M_PI/2)  // 1
```

Opis wybranych bibliotecznych funkcji matematycznych jest zawarty w tabeli 10.1.

Tabela 10.1. Opis wybranych funkcji matematycznych

Nazwa	Opis
`abs(argument)`	Zwraca wartość bezwzględną z `argumentu`
`sqrt(argument)`	Zwraca pierwiastek kwadratowy `argumentu`
`pow(podstawa, wykładnik)`	Zwraca wartość równą `podstawie` podniesionej do potęgi `wykładnik`
`log(argument)`	Zwraca wartość logarytmu naturalnego z `argumentu`
`exp(argument)`	Zwraca wartość funkcji wykładniczej: *e* podniesione do potęgi `argument`
`log10(argument)`	Zwraca wartość logarytmu dziesiętnego z `argumentu`
`sin(argument)`	Zwraca wartość funkcji *sinus* z `argumentu` mierzonego w radianach
`cos(argument)`	Zwraca wartość funkcji *cosinus* z `argumentu` mierzonego w radianach
`tan(argument)`	Zwraca wartość funkcji *tangens* z `argumentu` mierzonego w radianach
`ceil(argument)`	Zwraca najmniejszą wartość całkowitą nie mniejszą od wartości `argumentu` — zaokrąglenie w górę
`floor(argument)`	Zwraca największą wartość całkowitą, która jest nie większa od wartości `argumentu` — zaokrąglenie w dół
`trunc(argument)`	Zwraca najbliższą wartość całkowitą, która jest nie większa od wartości argumentu — obcięcie części ułamkowej
`round(argument)`	Zwraca wartość całkowitą, której wartość jest najbliższa wartości

Przykład 10.1 - co robi kod?

Program pokazuje, że po dołączeniu <cmath> można korzystać z wielu gotowych funkcji matematycznych. Stała M_PI wymaga w tym przykładzie dyrektywy _USE_MATH_DEFINES.

```
fmin(), fmax() i modf().

#include <iostream>
#define _USE_MATH_DEFINES // w celu użycia stałej M_PI, która nie jest zdefiniowana w standardowym
// Dołączenie biblioteki cmath do programu:
#include <cmath>
using namespace std;

int main() {
    cout << "Wartość stałej PI: " << M_PI << endl;

    cout << "Wartość bezwzględna liczby -1 = " << abs(-1) << endl;
    cout << "Kwadrat liczby 3 = " << pow(3, 2) << endl;
    cout << "Pierwiastek kwadratowy liczby 4 = " << sqrt(4) << endl;

    cout << "Mniejsza z dwóch liczb: 1 i 10 = " << fmin(1, 10) << endl;
    cout << "Większa z dwóch liczb: 1 i 10 = " << fmax(1, 10) << endl;

    cout << "Zaokrąglenie do liczby całkowitej liczby 1.55 = " << round(1.
         << endl;

    double czesc_calkowita;
```

- abs(-1) zwraca wartość bezwzględną
- fmin(1,10) i fmax(1,10) wybierają mniejszą lub większą liczbę
- modf(1.55, &czesc_calkowita) rozdziela część całkowitą i ułamkową
- argumenty funkcji trygonometrycznych są podawane w radianach

Funkcje znakowe - <cctype>

Funkcje znakowe sprawdzają lub przekształcają pojedyncze znaki typu char. Wyniki funkcji sprawdzających interpretujemy logicznie: prawda/fałsz.

`isalnum(argument)`	Sprawdza, czy `argument` należy do grupy znaków alfanumerycznych. Zwraca wartość `true`, jeśli znak jest literą (małą lub dużą) albo cyfrą. W przeciwnym razie zwraca `false`
`isalpha(argument)`	Sprawdza, czy `argument` jest literą alfabetu, z uwzględnieniem ustawień narodowych. Jeśli tak, zwraca wartość `true`, jeśli nie — `false`
`islower(argument)`	Sprawdza, czy `argument` jest małą literą. Jeśli tak, zwraca wartość `true`, jeśli nie — `false`
`isupper(argument)`	Sprawdza, czy `argument` jest dużą literą. Jeśli tak, zwraca wartość `true`, jeśli nie — `false`
`isdigit(argument)`	Sprawdza, czy `argument` jest cyfrą dziesiętną. Jeśli tak, zwraca wartość `true`, jeśli nie — `false`
`isxdigit(argument)`	Sprawdza, czy `argument` jest cyfrą szesnastkową. Jeśli tak, zwraca wartość `true`, jeśli nie — `false`
`isprint(argument)`	Sprawdza, czy `argument` jest znakiem drukowanym (ang. *printable character*). Jeśli tak, zwraca wartość `true`, jeśli nie — `false`
`ispunct(argument)`	Sprawdza, czy argument jest znakiem interpunkcyjnym (ang. *punctuation character*). Jeśli tak, zwraca wartość `true`, jeśli nie — `false`
`iscntrl(argument)`	Sprawdza, czy `argument` jest znakiem sterującym (ang. *control character*). Jeśli tak, zwraca wartość `true`, jeśli nie — `false`. Działanie odwrotne do funkcji `isprint()`

```
char znak = 'c';
cout << isalnum(znak);
cout << isalpha(znak);
cout << islower(znak);
cout << toupper(znak);
```

- isalnum - litera lub cyfra
- isalpha - litera
- isdigit - cyfra
- islower/isupper - wielkość litery
- tolower/toupper - konwersja wielkości

Konwersja typu danych - <cstdlib>

Konwersja jest potrzebna, gdy dane tekstowe trzeba zamienić na liczbę. W podręczniku omawiane są funkcje konwertujące C-napisy, czyli napisy typu const char*.

W programie często trzeba wykonywać różnego rodzaju operacje pomocnicze, takie jak konwersja łańcucha znaków na liczbę, sortowanie tablicy czy wygenerowanie liczby losowej

Funkcje ogólnego przeznaczenia umożliwiające realizację tych i innych operacji są zgrupowane w bibliotece *cstdlib* (w języku C — *stdlib.h*).

Wykaz wybranych funkcji zawartych w bibliotece *cstdlib* przedstawiono w tabeli 10.3.

Tabela 10.3. Opis wybranych funkcji standardowych biblioteki cstdlib

Nazwa	Opis
`atof(argument)` `strtod(argument, NULL)`	Konwersja C-napisu `argument` typu `const char*` na liczbę zmiennoprzecinkową należącą do typu `double`. Funkcje zwracają wartość liczby typu `double`
`strtof(argument, NULL)`	Konwersja C-napisu `argument` typu `const char*` na liczbę zmiennoprzecinkową należącą do typu `float`. Funkcja zwraca wartość liczby typu `float`

```
const char *napisI = "3";
const char *napisF = "3.14159";

atoi(napisI)
strtol(napisI, NULL, 0)
atof(napisF)
strtod(napisF, NULL)
strtof(napisF, NULL)
```

Ćwiczenie: skonwertuj tekst "2.718281828459045235536" na double i wypisz wynik zaokrąglony do 4 miejsc po przecinku.

Funkcje wejścia/wyjścia - <cstdio>

C++ zwykle używa cin i cout, ale można także korzystać z funkcji odziedziczonych z C: scanf() do wczytywania i printf() do formatowanego wyświetlania.

```
float bok1, bok2, pole;
printf("Pierwszy bok = ");
scanf("%f", &bok1);

pole = bok1 * bok2;
printf("Pole wynosi: %6.2f", pole);
```

```
%f - float/double w printf
%d - liczba całkowita
%c - pojedynczy znak
%s - C-napis
%6.2f - szerokość 6, 2 miejsca po przecinku
```

```
float bok1, bok2, pole, obwod;

// Wyświetlenie komunikatu na ekranie monitora:
printf("Podaj długości boków prostokąta: \n");
// Wyświetlenie komunikatu:
printf("Pierwszy bok = ");

// Wprowadzenie wartości zmiennej bok1 z klawiatury:
scanf("%f",&bok1);

printf("Drugi bok = ");
scanf("%f",&bok2);

pole = bok1 * bok2;

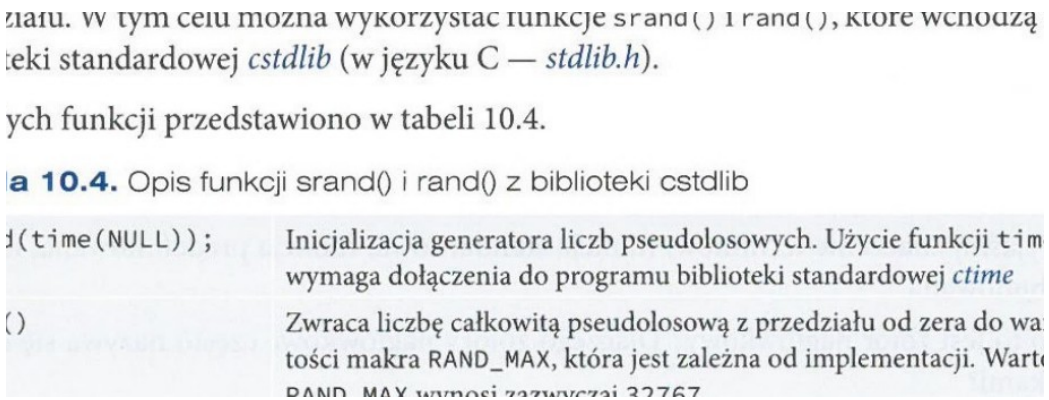
// Wyświetlenie komunikatu pomocniczego i wartości zmiennej pole na ekranie monitora:
printf("Pole wynosi: %6.2f\n",pole);

obwod = 2 * bok1 + 2 * bok2;
// Wyświetlenie komunikatu i wartości zmiennej obwod na ekranie monitora:
printf("Obwod wynosi: %6.2f\n",obwod);

return 0;
```

Liczby pseudolosowe - srand(), rand(), time()

rand() zwraca liczbę pseudolosową, a srand(time(NULL)) inicjalizuje generator inną wartością startową przy uruchomieniu programu.



ziału. W tym celu można wykorzystać funkcje `srand()` i `rand()`, które wchodzą
teki standardowej *cstdlib* (w języku C — *stdlib.h*).

ych funkcji przedstawiono w tabeli 10.4.

a 10.4. Opis funkcji srand() i rand() z biblioteki cstdlib

`d(time(NULL));`	Inicjalizacja generatora liczb pseudolosowych. Użycie funkcji `tim` wymaga dołączenia do programu biblioteki standardowej *ctime*
`()`	Zwraca liczbę całkowitą pseudolosową z przedziału od zera do wa tości makra `RAND_MAX`, która jest zależna od implementacji. Wart `RAND_MAX` wynosi zazwyczaj 32767

```
#include <ctime>
#include <cstdlib>

srand(time(NULL));
cout << RAND_MAX << endl;
cout << (rand() % 100 + 1); // 1..100
```

Wzór na przedział 100-1000: rand() % 901 + 100

Nowe skróty i pojęcia z lekcji

STL Standard Template Library - część biblioteki standardowej C++ z kontenerami i algorytmami.

C-napis Łańcuch znaków w stylu C, zwykle reprezentowany jako const char*.

header file Plik nagłówkowy, np. <cmath>, zawierający deklaracje funkcji.

RAND_MAX Maksymalna wartość, jaką może zwrócić rand().

NULL Specjalna wartość wskaźnikowa używana m.in. w przykładach konwersji.

printf/scanf Funkcje wejścia/wyjścia z biblioteki <cstdio>.

Zadania praktyczne

- Napisz program, który oblicza sqrt(x), x^2 oraz |x| dla liczby podanej z klawiatury.
- Sprawdź, czy podany znak jest literą, cyfrą, znakiem drukowalnym i zamień go na wielką literę.
- Skonwertuj C-napis z liczbą Eulera na double i wyświetl go z dokładnością do 4 miejsc po przecinku.
- Zmień program z prostokątem tak, aby dane były typu int, a wynik wyświetlany przez printf().
- Wygeneruj liczbę pseudolosową z przedziału 100-1000.

Uczymy się nie tylko zapamiętywać nazwy funkcji, ale dobierać właściwy nagłówek i typ danych do problemu.

Podsumowanie

<cmath> <cctype> <cstdlib> <cstdio> <ctime>

- Funkcje biblioteczne przyspieszają pracę i zmniejszają liczbę błędów.
- Aby użyć funkcji, trzeba znać jej nagłówek, argumenty i typ wyniku.
- Funkcje matematyczne często wymagają argumentów w radianach.
- printf() i scanf() wymagają poprawnych specyfikatorów formatu.
- rand() generuje liczby pseudolosowe, a nie prawdziwie losowe.

Exit ticket: Wyjaśnij różnicę między funkcją pow(), funkcją round() i operatorem %, a następnie podaj po jednym przykładzie zastosowania.